

Anomaly Detection in Event-Triggered Traffic Time Series via Similarity Learning

Shaoyu Dou , Kai Yang , Senior Member, IEEE, Yang Jiao , Chengbo Qiu , and Kui Ren , Fellow, IEEE

Abstract—Time series analysis has achieved great success in cyber security such as intrusion detection and device identification. Learning similarities among multiple time series is a crucial problem since it serves as the foundation for downstream analysis. Due to the complex temporal dynamics of the event-triggered time series, it often remains unclear which similarity metric is appropriate for security-related tasks, such as anomaly detection and clustering. The overarching goal of this paper is to develop an unsupervised learning framework that is capable of learning similarities among a set of event-triggered time series. From the machine learning vantage point, the proposed framework harnesses the power of both hierarchical multi-resolution sequential autoencoders and the Gaussian Mixture Model (GMM) to effectively learn the low-dimensional representations from the time series. Finally, the obtained similarity measure can be easily visualized for the explanation. The proposed framework aspires to offer a stepping stone that gives rise to a systematic approach to model and learn similarities among a multitude of event-triggered time series. Through extensive qualitative and quantitative experiments, it is revealed that the proposed method outperforms state-of-the-art methods considerably.

Index Terms—Anomaly detection, clustering, event-triggered time series, Internet of Things security, software security.

I. INTRODUCTION

TIME series anomaly detection is crucial in various domains, including cyber-security and astronomy. A central challenge in time series anomaly detection is measuring the *similarity* between two time series, which is essential for comparing samples and differentiating abnormal from normal. Practically, computing a suitable similarity metric lies at the heart of numerous machine learning tasks, including clustering and supervised classification. The motivation for learning the similarity of *event-triggered time series* in this paper is to detect anomalous or malicious behavior of devices or software via their traffic. For instance, a hacked malicious healthcare IoT

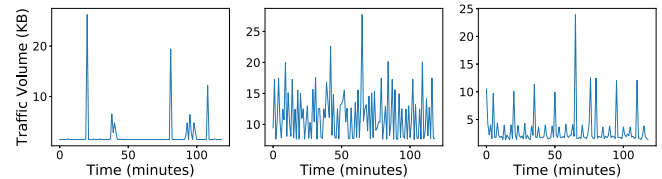


Fig. 1. Representative event-triggered traffic time series from the UNSW-IoT dataset.

device may send sensitive personal information to the Internet, which compromises users' privacy and requires immediate attention [1]. More generally, machine learning applied to time series, including applications like network device behavior detection and IoT detection, is anticipated to be pivotal in various emerging applications [2], [3], [4], [5]. This motivates the extension of similarity learning beyond just anomaly detection to a broader range of machine learning tasks, such as clustering.

The past decade has witnessed a proliferation of event-triggered sensors or software-generated time series data, where events refer to human intervention or programmed machine activity. Such events will trigger the working state transition of the program, resulting in heterogeneous dynamics of the traffic time series. Please refer to Section III-A for an informal definition of event-triggered time series. A key challenge in analyzing such event-triggered (traffic) time series is that it often contains temporal event sequences that are sporadic or highly heterogeneous as shown in Fig. 1. It may contain a few short traffic bursts and a long sleep time with no data transmission, as shown in the first sub-figure, or seems to be the “superposition” of multiple time series, as shown in the second and third sub-figures. The unique pattern makes this type of time series extremely heterogeneous and exhibits both long-term and short-term temporal dependencies which render the traditional machine learning algorithms not directly applicable. Apart from the heterogeneity, other challenges include: 1) Since the labeling process is often very expensive, there are rarely sufficient and accurate labels for similarity learning. 2) To achieve the best performance, similarity learning needs to be tuned for a particular task. 3) Event-triggered sensors or software-generated traffic time series data often vary in time granularity. 4) In many applications, we need to not only compute the similarity between two unlabeled time series but also provide insight into the mechanism so that domain experts can understand the similarity metric.

Manuscript received 5 December 2022; revised 28 March 2024; accepted 16 June 2024. Date of publication 25 June 2024; date of current version 14 March 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 12371519, Grant 61771013, and Grant 62032021, in part by the Fundamental Research Funds for the Central Universities of China, and in part by the Fundamental Research Funds of Shanghai Jiading District. (Corresponding author: Kai Yang.)

Shaoyu Dou, Kai Yang, Yang Jiao, and Chengbo Qiu are with the Department of Computer Science and Technology, Tongji University, Shanghai 201800, China (e-mail: kaiyang@tongji.edu.cn).

Kui Ren is with the Zhejiang University, Hangzhou 310027, China (e-mail: kaiyang@tongji.edu.cn).

Digital Object Identifier 10.1109/TDSC.2024.3418906

The above challenges give rise to the following questions.

How can we design an unsupervised machine learning approach to learn the similarity between two event-triggered time series?

The traditional sequential autoencoder, such as GRU or LSTM-based autoencoder, encounters the problem of error accumulation during autoregressive decoding [6], [7], leading to suboptimal reconstruction and representation quality, particularly for non-smooth time series with high temporal dynamics, as shown in Fig. 1. In this paper, we present ET-Net, a bagging model designed to effectively model the temporal dynamics of Event-Triggered time series. To address the challenges mentioned above, we propose two ensemble components that adopt different autoencoders, which are named W compression network and D compression network and inspired by multi-task and multi-resolution learning, respectively. The proposed autoencoders learn robust representations and form the basis of similarity learning. Each ensemble component then utilizes a statistical Gaussian Mixture Model (GMM) to measure similarities among a collection of time series on the learned representations. The overall ET-Net provides visual outcomes to aid human understanding. More specifically, we have made the following contributions:

- *Unsupervised similarity learning:* ET-Net is completely unsupervised and can learn similarities among unlabeled event-triggered time series. In addition, similarity learning can be tuned for a particular task to optimize the detection performance.
- *Visualization and interpretability:* ET-Net generates a semantically meaningful latent space that can be visualized to explain the learned model. In addition, some classification rules can be drawn from the visualization of the original space to help human experts understand the data.
- *Effectiveness:* ET-Net exhibits strong empirical performance for downstream analysis such as anomaly detection and clustering than other competing state-of-the-art methods over real-world datasets. It remains effective even when the training data is contaminated by anomalies or noises that are common in time series analysis [8]. In addition, the proposed model still has competitive performance on testing data with different time granularity [9] and traffic disturbances without retraining.

II. RELATED WORK

A. Similarity Learning

We summarize related work on similarity learning that has received significant attention over the past decade, including non-parametric methods and parametric methods based on deep neural networks.

There exists a large body of work on non-parametric methods for time series similarity learning, including euclidean Distance (ED) [10], Editing Distance on Real sequences (EDR) [11], and Dynamic Time Warping (DTW) [12]. Euclidean distance, while the most widely used distance metric, often yields poor performance for time series similarity learning because it is sensitive to anomalies, noise, warping and contamination [13].

To address this issue, one may employ techniques such as Edit Distance with Real Penalty (EDR) and Dynamic Time Warping (DTW) to handle sequences of unequal length. However, EDR is calculated based on local procedures and treats all change operations equally, making it sensitive to noise and irregular sampling rates. Thus, even minor deviations in a group of data points within the time series can cause EDR to produce large values, compromising its effectiveness. DTW is a time series similarity metric that has been widely studied and used in recent years. It aims to calculate an optimal matching between two time series and has proved to be capable of providing strong baseline performance in many machine learning tasks such as classification and clustering. In addition, there exists a lot of work dedicated to improving the performance of DTW [14] or combining DTW with deep learning methods [15], [16].

The majority of deep learning-based methods for time series similarity learning use a Recurrent Neural Network (RNN) or Convolutional Neural Network (CNN) to model the temporal dynamics and convert them into low-dimensional representations. Similarity or distance metrics in this low-dimensional latent space are expected to reflect the semantic relationship between time series. [17] proposes a structure called WaRTEM to generate time series embedding that exhibits resilience to warping. [18] proposes a model named DTCR that integrates the seq2seq model and the K-means objective to generate latent space representations that are better suited for clustering. Auto-warp proposed in [19] obtains a vector embedding through the sequence autoencoder, which helps to guide the optimization of a warping metric.

B. Anomaly Detection

Time series anomaly detection can be broadly categorized as either supervised or unsupervised methods. Supervised anomaly detection techniques such as Support-Vector Machine (SVM) [20] and Random Forest (RF) [21] require a large amount of accurate labels to achieve optimal performance. However, in practice, imbalanced training data and inaccurate labeling can lead to performance degradation. Additionally, in many cases, it is not possible to obtain labels for anomaly detection, making the supervised approach completely inapplicable. Unsupervised anomaly detection, which includes classification-based methods [22] and density-based methods [23], [24], detects anomalies by training a one-class classifier on normal data points or by performing density estimation. Although these methods can achieve satisfactory anomaly detection accuracy, they are often ineffective when dealing with high-dimensional time series data.

Recently, reconstruction-based deep learning methods have emerged as a new means for unsupervised anomaly detection in high-dimensional data. These methods assume that the reconstructions of low-dimensional projections of anomalies will deviate greatly from the original samples, and use the reconstruction error to detect anomalies [25], [26]. However, vanilla reconstruction-based methods are often limited because they only conduct anomaly detection based on reconstruction error. Hybrid methods have been developed to harness the power of both autoencoders and model-driven approaches. This

combined approach compresses the original data into a latent space and then performs model estimation in that space. For instance, [18] uses density-based K-means clustering in the latent space, while [27] and [28] model the latent space from the perspective of probability by leveraging normal distribution and Gaussian Mixture Model (GMM), respectively. However, the assumption that the reconstruction error follows a Gaussian distribution in [27] is not applicable to highly heterogeneous event-triggered time series. This is because different networks have varying reconstruction performances on heterogeneous data. Additionally, [29] learns a normalized flow in the latent space.

It is worth noting that some recent studies have explored hybrid methods or ensembled autoencoders which are also adopted in proposed ET-Net. For instance, [30] proposed a two-stage approach that trains an SAE-based representation network with GMM, which can lead to suboptimal results due to the separate optimization. In contrast, ET-Net adopts an end-to-end approach to jointly optimize the representation learning and density estimation. As a result, the representations are constrained by GMM, leading to a visually interpretable latent space. Another example is [31], which proposed learning graph representations with a node similarity measure based on topology as supervision, but this type of supervised information is not available for time series since there is no prior knowledge that represents the similarity among a set of time series. In [32], sample representations are generated using a homogeneous multichannel autoencoder, and the network is optimized in a supervised manner using domain labels. In contrast, ET-Net is a completely unsupervised architecture that learns the similarity of event-triggered time series without relying on any prior knowledge or labels.

C. Visual Interpretability

Visual interpretability is often seen as the first step in explaining deep neural networks. It can be used to explain the inherent mechanism of the neural network [18], [28], [33], and can also be used to trace which training samples or features significantly affect the output of the neural network, i.e., attributing the output of neural network to a set of features or samples. In general, the attribution methods can be roughly categorized into two groups, i.e., feature-based and example-based. The feature-based attribution methods compute the contribution of each input feature to the model output and visualize the result through a heat map superimposed on the input sample. LIME [34] optimizes a white-box model to locally approximate the output of the given neural network, and then determine the contribution of each feature by analyzing the learned white-box model. SHAP [35] calculates the contribution of each feature based on game theory. [36] optimizes a masking model to identify the input features that most influence the decision of the classifier. IntegratedGrads [37] proposes to use the integrated gradients of the feature as the importance score of the feature. However, such methods often struggle to generate convincing attribution results on time series data, because the model may highlight features that the model considers important but not to human experts [38].

Example-based attribution methods visualize a set of training samples or prototypes to explain the output of the network. [39] utilizes the influence function to determine which training samples play a decisive role in the model prediction for a given sample. [40] proposes that the K nearest neighbors of a given sample in the feature space are the training samples that contribute the most to the network output. While extensive efforts have been undertaken for the example-based attribution method, its application to time series data is still an under-explored area.

ET-Net adopts an example-based visually interpretable approach for several reasons. First, feature-based approaches such as LIME and SHAP attribute the model output to a set of data points in a time series. However, in event-triggered time series, each data point is aggregated from the behavior of multiple events, thus analyzing only a single sample, and attributing output to data points does not explore the specific event that caused the anomaly, nor help humans understand how to distinguish normal and abnormal samples. In contrast, the example-based method provides a group of similar abnormal samples and a set of normal samples for comparison when analyzing a given abnormal sample. Although this cannot directly indicate the specific reason for the anomaly, humans can obtain knowledge to distinguish normal and abnormal samples by comparing and summarizing these samples, as discussed in Section IV-C4. Moreover, while some methods, such as the counterfactual sample generation-based method [41], add perturbations to the original sample to generate samples that would alter the model prediction, this can be problematic in time series analysis scenarios. Specifically, any artificial perturbation on the time series can cause difficulties understanding the time series. Furthermore, interpretation methods designed for specific network structures, such as CNN [42], can be challenging to extend to the task of time series analysis. To avoid the drawbacks mentioned above, ET-Net employs a similar approach as [40], but instead outputs normal and abnormal samples for a given anomaly sample in an unsupervised setting. This approach helps humans to better understand the anomaly and may summarize knowledge from that.

In this paper, we propose an end-to-end general framework for learning the similarity between event-triggered time series in a fully unsupervised manner. The resulting outcomes can be easily visualized in latent space for human comprehension. We also use the example-based attribution method to explain the model decisions from the original space. Moreover, the proposed model learns the vector embeddings in the latent space by taking into account the machine learning task under investigation. Finally, the probabilistic GMM model offers probabilistic measurement and is more flexible than the K-means clustering method. We summarize the unique features of our model and compare it with other state-of-the-art approaches in Table I. In particular, the GMM adopted in this framework adopts mixed membership and is much more flexible in terms of cluster covariance than the hard assignment approach such as the K-means clustering method [18]. The proposed task-aware approach can learn the vector embeddings that are tailored for a particular machine learning task, so the detection performance can be improved.

TABLE I
COMPARISON OF RELATED WORK

	ED	DTW	EDR	WaRTEm [17]	DTCR [18]	Autowarp [19]	ET-Net
Robustness	×	✓	×	✓	✓	✓	✓
Compression	×	×	×	✓	✓	✓	✓
Task awareness	×	×	×	×	×	×	✓
Flexibility	×	×	×	×	×	×	✓
Joint learning	×	×	×	✓	✓	×	✓

TABLE II
CHARACTERISTICS OF MTC AND HTC EVENTS

Event Type	Volume	Pattern
MTC	Small	Almost periodic Short transmission period
HTC	Large	Bursty and unpredictable Generally long transmission interval

As evident from Table I, only ET-Net meets all the desired requirements.

III. HIERARCHICAL MULTISCALE VARIATIONAL AUTOENCODER WITH GAUSSIAN MIXTURE MODEL

A. Problem Statement

Event-triggered time series: Here we take the traffic time series of IoT devices as an example to illustrate the temporal characteristics of such event-triggered traffic time series.

Let $\mathbf{x}^\top \in \mathbb{R}^L$ denote a traffic time series of length L which is triggered by K events, i.e.,

$$\mathbf{x} = f(\mathbf{a}_1 \circ \mathbf{e}_1, \dots, \mathbf{a}_K \circ \mathbf{e}_K), \quad (1)$$

where $\mathbf{e}_i^\top \in \mathbb{R}^L$ is an indicator sequence, and its t th element, $e_{it} \in \{0, 1\}$, signifies whether event i is triggered at time t . $\mathbf{a}_i^\top \in \mathbb{R}^L$ represents an intensity sequence, and its t th element, $a_{it} \in \mathbb{R}$, denotes the traffic generated by event i at time t . The symbol $\circ$ denotes the Hadamard product. The events discussed in this paper typically fall into two categories: Machine-type communication (MTC) events, like transmitting sensing data and sending DNS requests, induce short-term and periodic dependencies. Meanwhile, Human Type Communication (HTC) events, such as requesting streaming videos or web pages via a smartphone, initiate long-term and bursty dependencies [43].

Consider the traffic generated by a webcam. The traffic time series of the webcam is a superposition of traffic from K events, represented as $\mathbf{x} = \sum_{i=1}^K \mathbf{a}_i \circ \mathbf{e}_i$, where interaction between the camera and controller, including routine queries and responses, are categorized as MTC events. These events, occurring frequently but consuming small traffic, can be considered background traffic, contributing to short-term dependencies in the time series. In contrast, human interactions, such as viewing surveillance videos, are classified as HTC events. While less frequent than MTC events, HTC events consume significant traffic and manifest as bursts in the traffic time series. The characteristics of MTC and HTC events are summarized in Table II.

Event-triggered time series are primarily characterized by *heterogeneity*, and *dependency across multiple time scales*. HTC events, originating from various applications and connecting to different servers, along with the unpredictability of human behavior, lead to significant heterogeneity even within the normal data. Furthermore, traffic from a single MTC event is notably homogeneous and typically periodic [43]. Due to the superposition of multiple events, the seasonality stemming from MTC events is observable across different time scales, resulting in an intricate dependency.

Problem Statement: Given a collection of event-triggered time series denoted by $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N]$, which may exhibit high dynamics, heterogeneity, and variation in time granularity. Our objective is twofold: (1) to obtain a robust low-dimensional representation vector for each time series in $\mathbf{X}$ for similarity measurement; (2) to detect anomalous time series $\mathbf{x}_i$ or to cluster the all time series based on the learned representations and similarity measurements.

We will describe the architecture of the basic ensemble component in Section III-B. Then, we introduce the W and D compression networks in Sections III-C and III-D, which are the main differences between the W and D branches. Other technical details are described in Section III-E.

B. General Framework

The basic ensemble component consists of two modules: a compression network and a distribution estimator. The compression network maps the event-triggered time series into a low-dimensional latent space $\mathcal{Z}$, while the distribution estimator models the probability distribution of the latent space using GMM. These two modules work in a coordinated manner to jointly capture the temporal dynamics of the event-triggered time series and generate vector embeddings optimized for GMM. The objective function used to learn this model is presented below.

$$L = \|\mathbf{X} - g(\mathbf{X})\|_2^2 + \lambda E(\mathbf{Z}, \mathbf{X}), \quad (2)$$

where $g(\cdot)$ is the autoencoder. $\mathbf{Z}$ is the representation of $\mathbf{X}$ in the latent space. $\|\cdot\|_2^2$ denotes the squared L2-loss. $E(\cdot)$ is the negative log-likelihood of the estimated GMM, a.k.a an energy function, which models latent space distribution. λ is a weighting parameter that governs the tradeoff between two individual objective functions. The above formula is similar to that of various existing works, such as VAE [44], DAGMM [28], DTCR [18], and SOM-VAE [45].

As previously mentioned, the heterogeneity and complex temporal dependencies in event-triggered time series pose challenges to representation learning. In this section, we propose

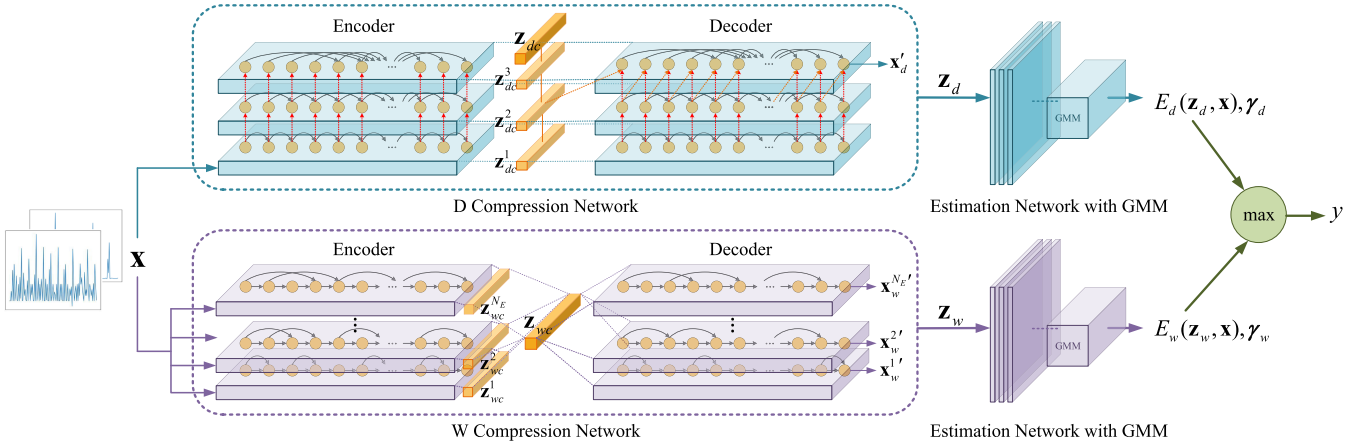


Fig. 2. The architecture of ET-Net.

two special types of sequence auto-encoder architectures that are partly similar to seq2seq [46] and are particularly suitable for learning the robust representation of event-triggered time series. We first propose the W compression network, which is inspired by multi-task learning, to tackle the heterogeneity. This network treats recurrent networks with varying time scales as independent tasks, jointly optimized for robust representation of heterogeneous sequences. To address the complex temporal dependencies, we developed a D compression network, inspired by multi-resolution learning. This network incrementally learns temporal representations, transitioning from fine-grained to coarse-grained. To avoid the curse of dimensionality, we equip each compression network with an estimation network featuring a learnable GMM, rather than directly concatenating the two representations. This approach constrains the sample within a GMM. We further ensemble these two anomaly detection models using the bagging method, aiming to enhance anomaly detection performance.

C. W Compression Network

The motivation behind designing a W compression network is to capture the intricate temporal dependencies among event-triggered time series in a multi-task learning fashion, which has been demonstrated to yield robust representations [47], [48]. This is particularly suitable for event-triggered time series, which can be highly heterogeneous. To achieve this, each encoder-decoder pair in the W compression network randomly models a different type of temporal dependency with a distinct time span using a Stochastic Recurrent Neural Network (SRNN) [49]. The output of the W compression network includes a compressed latent space representation $\mathbf{z}_{wc}$ and an extended latent space representation $\mathbf{z}_w$. The architecture of the W compression network is illustrated in Fig. 2.

Reconstructed sequence output by i th decoder $\mathbf{x}_w^{it}$ is computed as $\mathbf{z}_{wc}^{it} = g_{wc}^i(\mathbf{x})$ and $\mathbf{x}_w^{it} = g_{wd}^i(\mathbf{z}_{wc})$, where $g_{wc}^i(\cdot)$ and $g_{wd}^i(\cdot)$ are the i th encoder and decoder of W branch, $\mathbf{z}_{wc}^{it}$ is the final state of i th encoder. $\mathbf{z}_{wc} = \mathbf{W}_w[\mathbf{z}_{wc}^1, \dots, \mathbf{z}_{wc}^{N_E}]^T + \mathbf{b}_w$, where N_E is the number of encoders/decoders, $\mathbf{W}_w$ and $\mathbf{b}_w$

denote a trainable weight matrix and a bias vector, respectively. The extended latent space representation is then given by $\mathbf{z}_w = [\mathbf{z}_{wc}, d_{rel}(\mathbf{x}, \mathbf{x}_w^{p'}), d_{cos}(\mathbf{x}, \mathbf{x}_w^{q'})]$, where $d_{rel}(\cdot)$ and $d_{cos}(\cdot)$ are the reconstruction error, denoting the relative distance and cosine similarity, respectively, p and q are the indexes of auto-encoder branch with the minimum reconstruction relative distance and cosine distance, respectively.

The *recurrent function* used to update the hidden state of the recurrent cell in the i th layer of is as follows,

$$\begin{aligned} \mathbf{h}^i(t) &= \frac{w_1^i(t) \cdot f_{rnn}(\mathbf{h}^i(t-1), x(t)) + w_2^i(t) \cdot f'(\mathbf{h}^i(t-s^i), x(t))}{w_1^i(t) + w_2^i(t)} \\ \text{s.t. } w_1^i(t), w_2^i(t) &\in \{0, 1\}, w_1^i(t) + w_2^i(t) \neq 0 \end{aligned} \quad (3)$$

where $w_1^i(t)$ and $w_2^i(t)$ are randomly initialized weights. $f_{rnn}(\cdot)$ denotes a non-linear function including Long-Short Term Memory (LSTM) [50] or Gated Recurrent Unit (GRU) [51]. $f'(\cdot)$ denotes a linear operation. s^i is a parameter that controls the memory ability of SRNN. When s^i is small, SRNN tends to learn short-term dependencies. Otherwise, it will learn long-term dependencies. We set the parameter s^i of each encoder/decoder to a different value but no more than three, so that it tends to learn short-term dependencies in time series.

When LSTM is set as the recurrent cell, $f_{rnn}(\cdot)$ in (3) can be expanded as $f_{lstm}(\cdot)$,

$$\begin{aligned} f_{lstm}(\mathbf{h}(t-1), \mathbf{c}(t-1), x(t)) &= \mathbf{o}(t) \circ \mathbf{c}(t) \\ \mathbf{o}(t) &= \sigma(\mathbf{W}_o \cdot [\mathbf{h}(t-1), x(t)] + \mathbf{b}_o) \\ \mathbf{c}(t) &= \mathbf{f}(t) \circ \mathbf{c}(t-1) + \mathbf{i}(t) \circ \tilde{\mathbf{c}}(t) \\ \mathbf{f}(t) &= \sigma(\mathbf{W}_f \cdot [\mathbf{h}(t-1), x(t)] + \mathbf{b}_f) \\ \mathbf{i}(t) &= \sigma(\mathbf{W}_i \cdot [\mathbf{h}(t-1), x(t)] + \mathbf{b}_i) \\ \tilde{\mathbf{c}}(t) &= \tanh(\mathbf{W}_c \cdot [\mathbf{h}(t-1), x(t)] + \mathbf{b}_c) \end{aligned} \quad (4)$$

where $\mathbf{i}$, $\mathbf{o}$ and $\mathbf{f}$ are input gate, output gate and forget gate respectively. $\mathbf{c}$ is the memory. $\mathbf{W}_o$, $\mathbf{W}_f$, $\mathbf{W}_i$, $\mathbf{W}_c$, $\mathbf{b}_o$, $\mathbf{b}_f$, $\mathbf{b}_i$ and $\mathbf{b}_c$ are the parameters to be learned. The $f_{rnn}(\cdot)$ is defined

as $f_{gru}(\cdot)$ when GRU is set as the recurrent cell.

$$\begin{aligned} f_{gru}(\mathbf{h}(t-1), x(t)) &= (\mathbf{1} - \mathbf{u}(t)) \circ \mathbf{h}(t-1) + \mathbf{u}(t) \circ \tilde{\mathbf{h}}(t) \\ \mathbf{u}(t) &= \sigma(\mathbf{W}_u \cdot [\mathbf{h}(t-1), x(t)] + \mathbf{b}_u) \\ \tilde{\mathbf{h}}(t) &= \tanh(\mathbf{W}_h \cdot [(\mathbf{r}(t) \circ \mathbf{h}(t-1)), x(t)] + \mathbf{b}_h) \\ \mathbf{r}(t) &= \sigma(\mathbf{W}_r \cdot [\mathbf{h}(t-1), x(t)] + \mathbf{b}_r) \end{aligned} \quad (5)$$

where $\mathbf{u}$ and $\mathbf{r}$ are update gate and reset gate respectively, $\mathbf{W}_u$, $\mathbf{W}_h$, $\mathbf{W}_r$, $\mathbf{b}_u$, $\mathbf{b}_h$ and $\mathbf{b}_r$ are parameters to be learned.

D. D Compression Network

The purpose of the D compression network is to use multi-resolution learning to capture the complex long-term dependencies (introduced by HTC) and short-term dependencies (introduced by MTC) in event-triggered time series. To achieve this, multiple layers of dilated RNNs [52] are stacked sequentially to obtain deep representations of time series. The D compression network outputs a compressed vector $\mathbf{z}_{dc}$ and an extended latent representation $\mathbf{z}_d$ that can be used for further processing. Fig. 2 illustrates the architecture of the D compression network.

The D compression network learns representations of the time series at different levels of granularity, from fine-grained to coarse-grained. The final state of each encoder layer is denoted as $\mathbf{z}_{dc}^i$, and the overall compressed vector is computed as $\mathbf{z}_{dc} = \mathbf{W}_d \cdot [\mathbf{z}_{dc}^1, \dots, \mathbf{z}_{dc}^{N_L}]^T + \mathbf{b}_d$, where $\mathbf{W}_d$ and $\mathbf{b}_d$ are trainable weight and bias parameters, respectively. N_L is the number of resolution levels. The resulting $\mathbf{z}_{dc}$ is then used as the input for the decoder, which generates the reconstructed time series $\mathbf{x}'_d$. The extended representation $\mathbf{z}_d$ is then formed by concatenating $\mathbf{z}_{dc}$ with the relative distance $d_{rel}(\mathbf{x}, \mathbf{x}'_d)$ and the cosine similarity $d_{cos}(\mathbf{x}, \mathbf{x}'_d)$.

The hidden state of the recurrent cell in the i th layer of dilated RNN is updated as

$$\begin{aligned} \mathbf{h}^i(t) &= f(\mathbf{h}^{i-1}(t), \mathbf{h}^i(t-d^i)) \\ \mathbf{h}^0(t) &= x(t) \end{aligned} \quad (6)$$

where d^i denotes the dilation size in i th layer. The hidden state of layer i at time t only depends on the state at $t-d^i$ and the fine-grained state at layer $i-1$. In practice, we set 3 as the dilations in first layer, then an exponential growth strategy is used to set the dilation in the subsequent layer, that is, $d^i = 3^i$.

E. Estimation Network, Loss Function and Task-Aware Output

1) *Estimation Network*: After obtaining the extended latent space representation $\mathbf{z}_w$ or $\mathbf{z}_d$, it is input to a GMM estimator for density estimation, as illustrated in the following equation. To simplify the notation, we omit the subscripts and use $\mathbf{z}$ to represent the extended latent space representation generated by either the W or D compression network.

$$\gamma = g_m(\mathbf{z}) \quad \varphi_k = \sum_{i=1}^M \frac{\gamma_{ik}}{M} \quad \mu_k, \Sigma_k = \eta(\{[\mathbf{z}_i, \gamma_i]\}_{i=1}^M), \quad (7)$$

where K is the number of mixture components in GMM and M is the number of samples in mixture component k . $g_m(\cdot)$ is a membership estimator, and γ is a K -dimensional vector

representing the probability that sample $\mathbf{z}$ belonging to the k th mixture component. φ_k , μ_k and Σ_k are mixture probability, mean and covariance for k th mixture component, respectively. $\eta(\cdot)$ denotes a function for computing the mean and covariance. In practice, we use the iterative EM algorithm to update μ_k and Σ_k based on $\mathbf{z}$, instead of computing them directly using γ and φ like DAGMM.

Once we obtain the parameters of the GMM model, the sample energy function (c.f. (2)) can be calculated as follows,

$$\begin{aligned} E(\mathbf{z}, \mathbf{x}) &= -\log \left(\sum_{k=1}^K \varphi_k \theta(\mathbf{z} | (\mu_k, \Sigma_k)) \right) \\ \theta(\mathbf{z}, (\mu_k, \Sigma_k)) &= \frac{1}{\sqrt{|2\pi\Sigma_k|}} \exp \left(-\frac{(\mathbf{z} - \mu_k)^2}{2\Sigma_k} \right) \end{aligned} \quad (8)$$

where the $\theta(\mathbf{z}, (\mu_k, \Sigma_k))$ is the density function of Gaussian distribution $\mathcal{N}(\mu_k, \Sigma_k)$.

2) *Loss Function and Task-Aware Output*: During training, ET-Net is trained end-to-end using either normal or unlabeled data. It's assumed that the unlabeled data contains few anomalies, an assumption typically holds due to the rarity of anomalies. The loss function for the W branch and the D branch is given below, where N is the number of training samples.

$$\begin{aligned} L_w &= \frac{1}{NN_E} \sum_{i=1}^N \sum_{j=1}^{N_E} \|\mathbf{x}_i - \mathbf{x}_{wi}^{j'}\|_2^2 + \frac{\lambda}{N} \sum_{i=1}^N E_w(\mathbf{z}_{wi}, \mathbf{x}_i) \\ L_d &= \frac{1}{N} \sum_{i=1}^N \|\mathbf{x}_i - \mathbf{x}'_{di}\|_2^2 + \frac{\lambda}{N} \sum_{i=1}^N E_d(\mathbf{z}_{di}, \mathbf{x}_i). \end{aligned} \quad (9)$$

where $E_w(\cdot)$ and $E_d(\cdot)$ denote the energy functions of W and D branches, respectively.

During testing, the output of ET-Net y corresponds to the specific machine learning task we aim to carry out. For anomaly detection, y represents the anomaly score of the sample $\mathbf{x}$, and is calculated as $y = \max(E_w(\mathbf{z}_w, \mathbf{x}), E_d(\mathbf{z}_d, \mathbf{x}))$. In doing so, the network outputs the highest anomaly score to ensure high recall. For clustering or classification tasks, y represents the predicted label and is defined as $y = \arg \max_i (\max([\gamma_w, \gamma_d]^T))$, where γ_w and γ_d represent probabilistic GMM membership predicted by W and D branches respectively. Here we take the maximum of the two predicted probabilities since the resulting output by the sharper softmax distribution is preferred [53].

IV. EXPERIMENTS

Recall that the motivation of this paper is to detect anomalous or malicious behavior of devices or software via their traffic time series. Besides focusing on anomaly detection, it also explores the underlying fundamental problem of similarity learning. Consequently, the experiment considers two security-related machine learning tasks, i.e., anomaly detection and clustering. The applications of these two tasks in security management include intrusion detection and device identification. Furthermore, the clustering results also aim to validate the adaptability of similarity learned by ET-Net on tasks other than anomaly detection. We carry out the study to answer the following questions regarding the proposed approach.

- *Effectiveness*: Whether ET-Net outperforms the existing state-of-the-art anomaly detection and clustering methods?
- *Robustness*: Whether the trained model capable of being robust to noise, time granularity variations and potential traffic disturbance, which often occur during practical deployment?
- *Visualization*: Can we visualize and interpret the similarity metric learned by ET-Net?

A. Datasets and Experimental Design

1) *Datasets*: We first conduct anomaly detection and clustering experiments with synthetic datasets to visualize the latent space learned by ET-Net, which provides an intuitive interpretation of the model results.

We then conduct anomaly detection on several public, real-world traffic datasets. The UNSW-IoT¹ [54], cell traffic.² IoT23,³ and PowerCons dataset from the UCR time series classification archives⁴ are four event-triggered time series datasets.

The UNSW-IoT and IoT23 datasets originate from traffic data of IoT devices. These devices generate traffic only when they communicate, and remain in a dormant state at other times. The communication events include HTC and MTC. During the collection process, packets from all devices are captured and recorded, with each device being identified by a unique label. We divide the traffic time series into non-overlapping windows, each covering a 120-minute interval. Within each window, each data point represents the number of packets collected in one minute. For the UNSW-IoT dataset, the targeted anomalous events for detection are communications initiated directly by humans on non-IoT devices. These events pose potential security risks to private data in the sensing network [55], [56]. In the IoT23 dataset, our objective is to detect malicious attack events within the sensing network.

The cell traffic dataset is derived from traffic data on the cellular base station, which generates traffic only when users within the coverage area initiate communication activities. If no users are utilizing the mobile network during a certain period, the data will be zero. Time series data from a selected cell are considered non-anomalous. We create anomalies by injecting traffic time series from another cell. Thus, anomalous events are defined as communications that deviate from a cell's historical patterns, possibly signifying unusual public events.

For the UNSW-IoT, IoT23, and cell traffic datasets, we divided them into training and test datasets with a 40-60 split. The proportion of normal and abnormal samples in these datasets are 0.012, 0.093, and 0.077, respectively.

The PowerCons dataset originates from household electricity usage, recording data solely during human electricity use. In this dataset, data from one season is considered normal, while data from other seasons are treated as anomalies [57]. Anomalous events in this dataset are defined as consumption behaviors that

deviate from a season's typical pattern, potentially indicating illegal activities such as unauthorized bitcoin mining.

In addition, we selected three non-event-triggered datasets from the UCR time series classification archive, MedicalImages, SmoothSubspace, and MoteStrain, as these datasets have similar heterogeneous properties as event-triggered time series. Additionally, the results on these datasets emphasize the adaptability of ET-Net to a wide range of applications. We followed the anomaly injection experimental setup detailed in [57] for these datasets, as well as for the PowerCons dataset. This approach ensures the abnormal-to-normal sample ratio does not exceed 0.1. AUC (Area under the receiver operating curve) is employed to assess the anomaly detection performance.

Likewise, the clustering performance of the proposed framework is elucidated via experiments on three real-world datasets, including UNSW-IoT, IoT23 and cell traffic. NMI (Normalized Mutual Information) is employed to assess the clustering performance.

2) *Baselines*: For anomaly detection, we compare ET-Net against the following state-of-the-art unsupervised methods, including One-Class SVM (OCSVM), Local Outlier Factor (LoF), Isolation Forest (IF), Dynamic Time Warping (DTW), Kit-Net [58], GRU-AE [59], Shared-SRNN [49], DAGMM [28], and USAD [60].

The baseline algorithms for clustering include K-means, GMM, K-means+DTW, K-means+EDR, K-shape [61], DEC [62], IDEC [63], SPIRAL [64], DTC [65], and Autowarp [19].

For a fair comparison, all baseline methods use the parameter settings recommended by the authors.

B. Latent Space Visualization on Synthetic Datasets

In this section, we visualize the latent space learned by ET-Net on anomaly detection and clustering tasks to elucidate the underlying mechanism of the ET-Net, and provide an intuitive explanation for the model outcomes. Finally, we examine the robustness of latent space representation against data granularity variations and different types of noise.

To assess visualization results quantitatively, we utilize the average Silhouette Coefficients (SC) in clustering tasks. An SC closer to 1 indicates better performance, whereas an SC approaching -1 signifies poorer clustering. For the unsupervised anomaly detection task, ET-Net uses a GMM to group normal samples in the latent space and identifies samples that deviate from the normal cluster as anomalies. Ideally, the samples within the normal cluster should be highly cohesive and deviate from the scattered anomalous samples, so we use the Silhouette Coefficient Ratio (SCR) for normal and abnormal clusters to evaluate the visualization results. An SCR greater than 1, with higher values, indicates better performance, while an SCR close to 1 signifies ineffective separation between normal and abnormal clusters.

1) *Anomaly Detection*: We first assess the performance of the proposed framework via conducting machine learning tasks on a synthetic dataset, as shown in Fig. 3. This dataset consists of three non-anomalous time series samples, i.e., a sine wave, a square wave, and a triangle wave. We also create a total of five

¹<https://iotanalytics.unsw.edu.au/iottraces.html>

²<https://dandelion.eu/datagems/SpazioDati/telecom-sms-call-internet-mi>

³<https://www.stratosphereips.org/datasets-iot23>

⁴https://www.cs.ucr.edu/~eamonn/time_series_data_2018

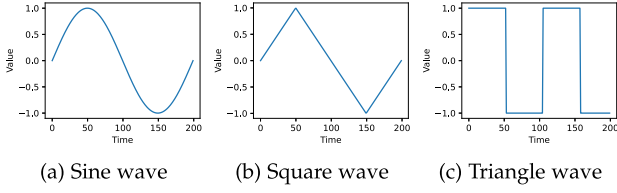


Fig. 3. Normal samples from the synthetic anomaly detection dataset that exhibit temporal heterogeneity.

hundred copies for each time series and use them to train the proposed deep learning model.

A total of four types of anomalies⁵ have been generated to assess the effectiveness of the proposed ET-Net framework. Type-1 anomaly refers to strong local additive white Gaussian noise, as shown in Fig. 4(a1). Type-2 anomaly stands for an unusually high activity that spans a short period of time (Fig. 4(b1)). Type-3 is the “breakdown” anomaly (Fig. 4(c1)). Type-4 anomaly is created by adding an impulse noise into the time series, as shown in Fig. 4(d1). Both the time domain and latent space representations are illustrated in Fig. 4. In the second and third sub-figures of Fig. 4, the blue symbols represent non-anomalous time series while the red ones correspond to anomaly time series. It is seen through both 2D and 3D visualization that ET-Net can effectively separate the anomalous time series from non-anomalous ones in the latent space.

Robustness against time granularity variations: We then explored the ability bounds of ET-Net to generate robust representations when the data granularity changes, taking a sin signal with a frequency of 10 Hz as an example, and Fig. 3(a) is an example of one of the cycles. We set the original sampling interval Δt_o to 1/120 second, and then vary the sampling rates from $\Delta t = 1/110, 1/130$ second to $\Delta t = 1/30, 1/5$ second, and visualize the obtained latent space representations in Fig. 5. Specifically, ET-Net produced robust time series representations when the data had small sampling frequency variations relative to sample 0 (e.g., 130 Hz, 110 Hz for samples 1 and 2). And when the sampling frequency is close to the Nyquist sampling frequency (20 Hz) for sample 3, compared to sample 4, which does not satisfy the sampling law, ET-Net generates a relatively more robust representation, as shown by point 3 being closer to point 0, 1 and 2 while point 4 being far from all data points. Next, we apply the obtained anomaly detection model to a test time series dataset in which the sampling rate differs from that of the training dataset. Fig. 6 illustrates the latent space representations of type-1 to type-4 anomaly time series and corresponding non-anomalous time series, where red and blue symbols represent abnormal and normal time series, respectively. It is seen that a ET-Net can be applied to time series with a different time granularity without any model retraining.

2) **Clustering:** This dataset consists of three types of time series, i.e., sine waves, square waves and triangle waves (as shown in different columns of Fig. 7), we also pass these time series through an Additive white Gaussian noise (AWGN) channel and introduce phase difference artificially (as shown in different rows

TABLE III
COMPARISON OF TIME SERIES SIMILARITY MEASURES [8], [66]

	ED	DTW	EDR	ET-Net
Increase sampling rate	Sensitive	Fair	Sensitive	Robust
Decrease sampling rate	Sensitive	Sensitive	Fair	Robust
Random shift	Robust	Robust	Robust	Robust
Add noise	Sensitive	Sensitive	Robust	Robust

of Fig. 7) to make them more realistic. The vector embeddings in the latent space can be obtained for the time series through the ET-Net, as visualized in Fig. 8 using the t-SNE algorithm. It is evident that we can easily cluster these time series in the latent space.

Robustness against different types of noise: Four types of noise described in [8] are considered in this experiment. Type-1 and type-2 noise stand for increasing (Fig. 9(a)) and decreasing sampling rate (Fig. 9(b)), respectively. Type-3 noise is the shifting noise (Fig. 9(c)). Type-4 noise refers to adding Gaussian noise to the entire time series (Fig. 9(d)).

We then apply the four types of noise to a sine time series and compute the euclidean distance between the original time series and the time series with noise in both original and latent spaces. As shown in Fig. 10, for all four types of noise, the euclidean distance will increase quickly with the level of noise. In contrast, the proposed framework remains effective in the presence of all four types of noise and can mine the similarity between the original time series and the ones with noise. As a matter of fact, as shown in Table III, it is the only method that remains robust against all types of noise.

C. Anomaly Detection Results on Real-World Datasets

1) **Implementation Details:** We conduct experiments on a workstation with 2 NVIDIA Titan V GPUs. TensorFlow is employed to implement the proposed ET-Net framework.

ET-Net contains four types of hyperparameters, 1) the number of encoders or decoders N_E in the W compression network; 2) the number of layers N_L in the D compression network; 3) the number of neurons N_N in each encoder or decoder; 4) the number of mixture components K in GMM. We perform a grid search to determine the hyperparameters of ET-Net. The hyperparameter settings for ET-Net are listed in Table IV. LSTM cell and GRU cell are adopted in the W and D compression network respectively. Please note that either GRU or LSTM can be used as the recurrent unit for both compression networks, in most cases, we recommend using GRU as the recurrent unit, considering the trade-off between model size and performance. We use the Adam optimization algorithm [67] to train the proposed model, and the initial learning rate is set to 10^{-3} .

2) **Effectiveness:** The performance of ET-Net and other state-of-the-art methods on a total of seven real-world datasets are

⁵<https://anomaly.io/anomaly-detection-twitter-r/>

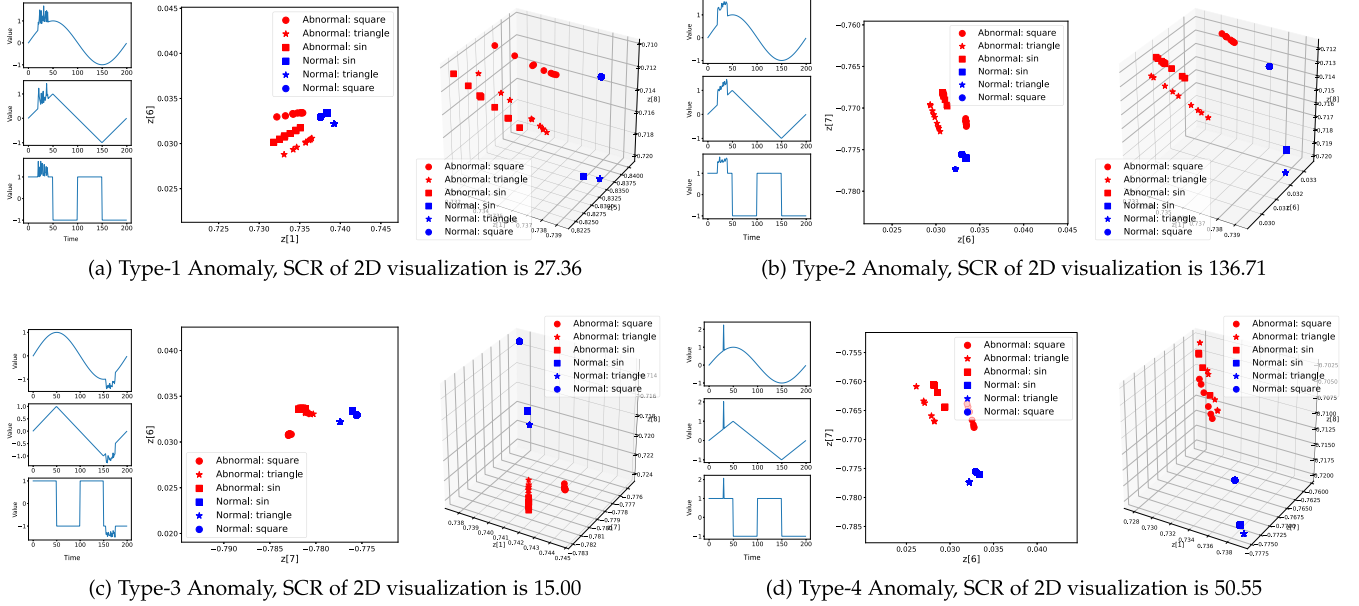


Fig. 4. Exemplary samples, 2D and 3D latent space visualizations of type-1 to type-4 anomalies. An SCR greater than 1, with higher values, indicates better visualization performance.

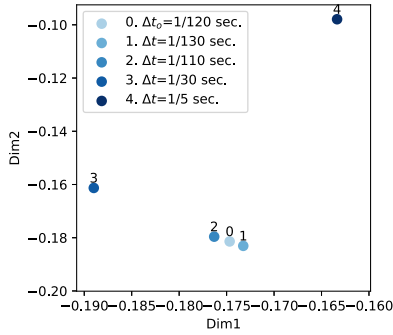


Fig. 5. Latent space representation of sine signals with different sampling intervals.

listed in Table V. ET-Net-W and ET-Net-D represent W compression network with GMM and D compression network with GMM, respectively. It is evident that the proposed ET-Net outperforms other competing methods considerably. It ranks first in five out of seven datasets, and ranks second in the remaining two datasets. In particular, for the cell traffic datasets, ET-Net outperforms the second-best method by around 14%. Furthermore, ET-Net outperforms ET-Net-W and ET-Net-D thanks to the ensemble of multiple networks. For the MoteStrain dataset, shared-SRNN and GRU-AE, based only on reconstruction errors, show superior detection performance compared to ET-Net-W and ET-Net-D, which integrate data representations with reconstruction errors. This is attributed to the severe shortage of training samples, that is, only 10 training samples, leading to the undertraining of large-parameter models, hindering the effective differentiation of normal and abnormal latent representations.

In the previous study, we assume all the training dataset constitutes non-anomalous time series. However, such an assumption

TABLE IV
HYPERPARAMETER SETTINGS

Dataset	N_E	N_L	N_N	K
Anomaly Detection				
UNSW-IoT	3	2	18	4
IoT23	3	2	18	4
Cell traffic	3	4	18	1
MedicalImages	3	2	8	3
MoteStrain	3	2	8	1
PowerCons	3	2	12	1
SmoothSubspace	3	2	8	1
Clustering				
UNSW-IoT	3	2	12	3
IoT23	3	4	18	8
Cell traffic	3	2	12	4

does not always hold in practice, since a small portion of the training data might be anomalies. As a remedy, we artificially inject anomalies into the training dataset to check whether we can still obtain an effective anomaly detector. Table VI demonstrates that the proposed ET-Net architecture remains effective even in the presence 10% of anomalies in the training dataset.

3) *Robustness Against Data Granularity and Traffic Disturbances*: Time series generated by event-triggered sensors can be highly complex due to different sampling intervals. This raises the question of whether an ET-Net model trained on time series with one sampling interval can be applied to a time series with other intervals. To investigate this, we trained an ET-Net model on a dataset with a 60-second sampling interval and tested its performance on time series with sampling intervals varying from 60 to 120 seconds. Table VII shows that the model remains effective even when the data granularity changes, indicating its

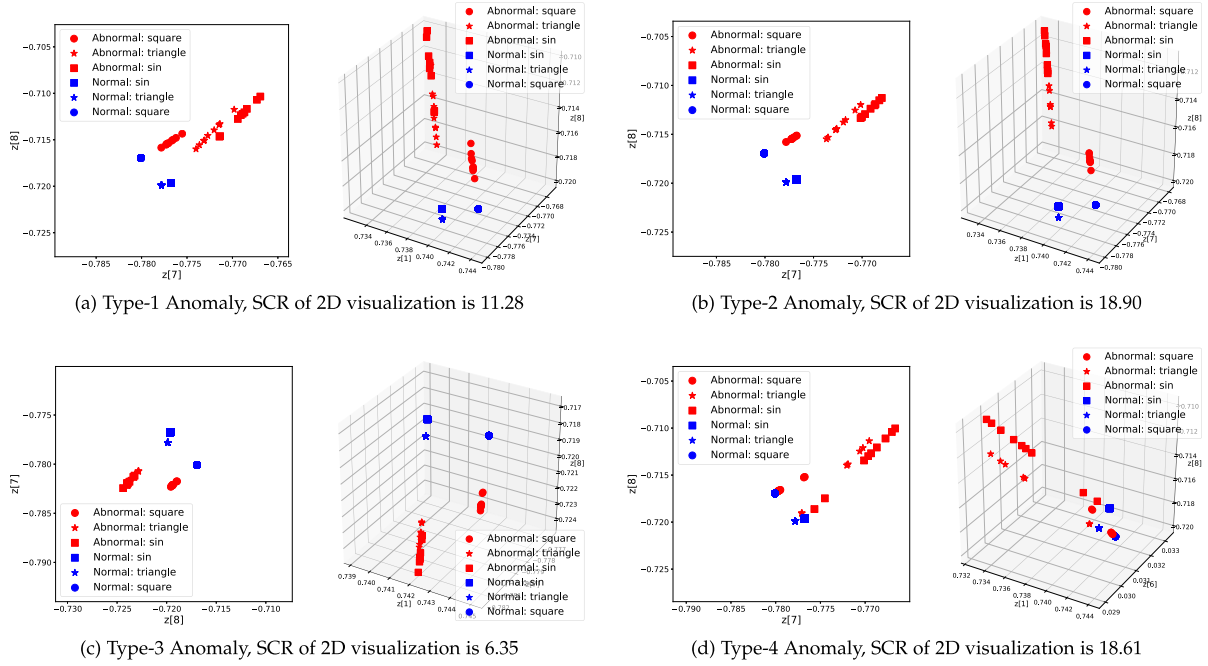


Fig. 6. 2D and 3D latent space representations of type-1 to type-4 anomalies with different sampling intervals. An SCR greater than 1, with higher values, indicates better visualization performance.

TABLE V
ANOMALY DETECTION PERFORMANCE MEASURED BY AUC

Method	UNSW-IoT	Cell traffic	IoT23	MedicalImages	MoteStrain	PowerCons	SmoothSubspace
OCSVM	0.7947	0.3241	0.5000	0.5366	0.8017	0.9827	0.9920
LoF	0.8705	0.5940	0.6402	0.6689	0.5339	0.7630	0.9200
IF	0.8586	0.4758	0.6544	0.5012	0.8959	0.9679	0.9920
DTW	0.8413	0.4865	0.6480	0.4508	0.8550	0.9642	1.0000
KitNET	0.9196	0.1667	0.2879	0.5074	0.7112	0.9975	1.0000
GRU-AE	0.9099	0.4259	0.7430	0.6358	0.9122	0.9691	0.9840
Shared-SRNN	0.8279	<u>0.6944</u>	0.7657	0.4680	0.9397	0.9716	0.9720
DAGMM	0.8314	0.5000	0.7964	0.6473	0.5000	0.5333	0.7800
USAD	<u>0.9384</u>	0.6482	0.3152	0.5782	0.8253	<u>0.9840</u>	<u>0.9960</u>
ET-Net-W	0.9356	0.3982	0.8504	<u>0.7584</u>	0.7597	0.9382	0.9840
ET-Net-D	0.8696	0.8333	0.6819	0.5724	0.9016	0.9506	1.0000
ET-Net	0.9503	0.8333	<u>0.8289</u>	0.7608	<u>0.9270</u>	0.9975	1.0000

Optimal and suboptimal results are bolded and underlined, respectively.

TABLE VI
AUC WITH INJECTED ANOMALIES IN TRAINING SET

UNSW-IoT		Cell traffic	
Proportions	AUC Score	Proportions	AUC Score
0%	0.9503	0%	0.8333
5%	0.9177	5%	0.8229
10%	0.9071	10%	0.7779
loss	4.32%	loss	5.54%

TABLE VII
AUC ON DIFFERENT SAMPLING INTERVALS

Sampling Intervals	60sec	90sec	120sec
OCSVM	0.7947	0.5010	0.5000
LoF	0.8705	0.8780	0.8664
IF	0.8586	0.8211	0.7939
DTW	0.8413	0.8511	0.8350
KitNET	0.9196	0.8501	0.8143
GRU-AE	0.9099	0.8903	0.8792
Shared-SRNN	0.8279	0.7541	0.7317
DAGMM	0.8314	0.7591	0.7494
USAD	<u>0.9384</u>	0.8803	<u>0.8930</u>
ET-Net	0.9503	0.8909	0.9078

Optimal and suboptimal results are bolded and underlined, respectively.

robustness. We also tested the robustness of ET-Net to traffic disturbances that may occur in real-world scenarios. We applied the trained model to a test dataset with different perturbation

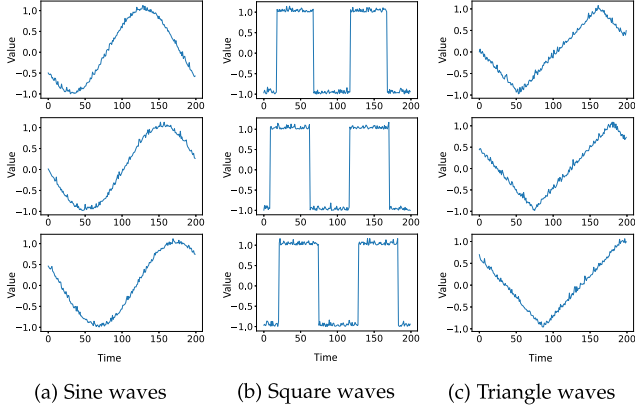


Fig. 7. Three category of samples from the synthetic clustering dataset.

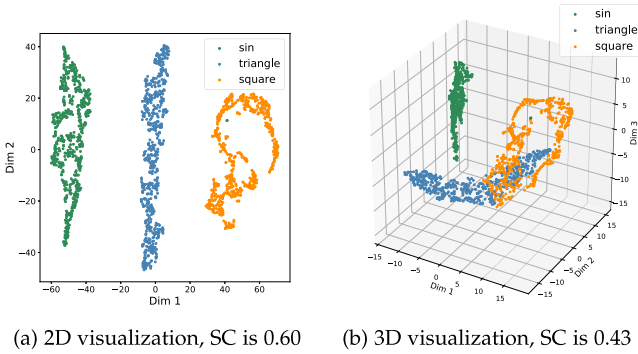


Fig. 8. 2D and 3D visualization of clustering. An SC closer to 1 indicates better performance, whereas an SC approaching -1 signifies poorer clustering.

ratios, the injected disturbances include adding dummy data packets [68] and adversarial perturbations [69]. Fig. 11 shows that even in the worst case where half of the samples are perturbed, the AUC of ET-Net is only reduced by about 2%, which is still better than the majority of baselines, demonstrating its robustness.

4) *Visualization and Interpretability*: From a perspective of latent space visualization, ET-Net models the distribution of normal samples using a GMM model, and forms a normal cluster in the latent space in the anomaly detection task. Thus, the vector embedding that deviates from this distribution is deemed as an anomaly. A typical example is shown in Fig. 4.

Based on the fact that normal samples are grouped into clusters in the latent space, we propose an *example-based attribution method* to explain the detected anomalies. Specifically, given a time series $\mathbf{x}_a$ that is deemed as an anomaly, we draw a straight line in latent space from the representation of $\mathbf{x}_a$ to the center of normal cluster $\mathbf{z}_{cnt}$. We call this line reference line hereafter. The comparison among the anomaly time series and corresponding reference time series around the reference line helps to explain the difference between the anomalous times series and the normal ones. Note that the reference samples are selected from the training set. Figures in the first column in Fig. 12 illustrate three representative abnormal time series, and

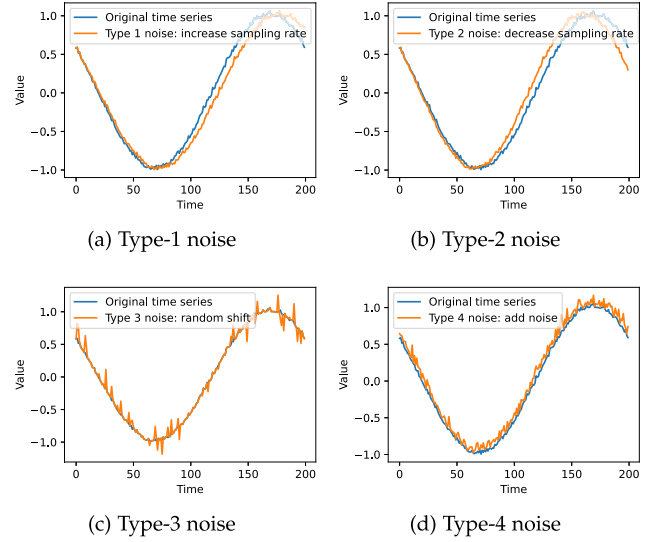


Fig. 9. Exemplary synthetic samples of four types of noise, using a sine signal as an example.

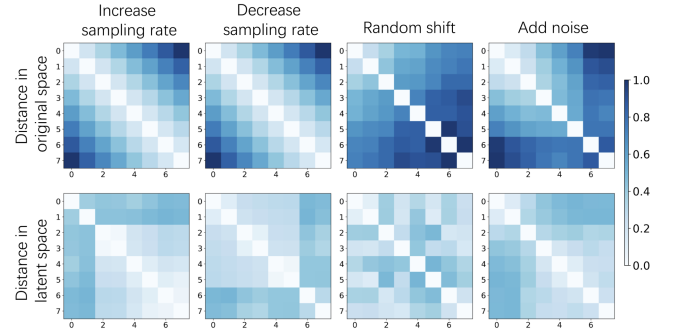


Fig. 10. Normalized euclidean distance matrix of a collection of time series in the original space (top row) and latent space (bottom row). The four columns are distance matrices when the time series contains different types of noise.

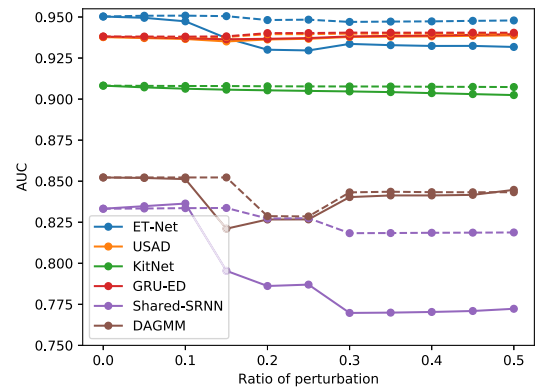


Fig. 11. AUC with perturbation in the testing set, where the solid and dashed lines represent adding adversarial perturbations and dummy data packets, respectively.

the remains are reference time series, where the samples in the second column are the reference sample closest to the abnormal samples, and the third and fourth columns of samples are closer to $\mathbf{z}_{cnt}$. See Fig. 14 for the complete figure. By observing these

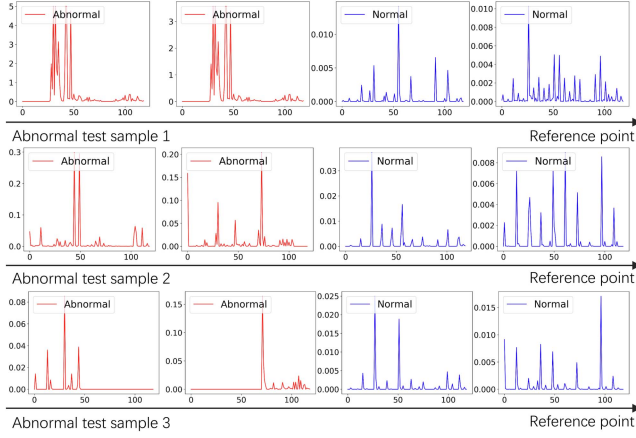


Fig. 12. Exemplary test samples and corresponding reference samples from the UNSW-IoT dataset. The x -axis represents time in minutes and the y -axis represents traffic in kilobytes.

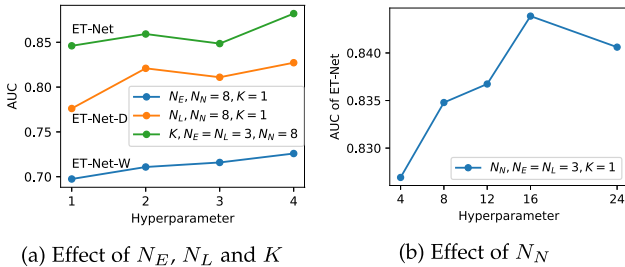


Fig. 13. Effect of hyperparameters.

examples, we may extract semantic information that may explain the difference between the abnormal and normal time series.

- Anomalous traffic time series may carry an unusually high amount of traffic data compared with normal traffic time series, as given in the first two examples.
- Abnormal traffic time series may bear long and deep sleeping modes in which no traffic is transmitted.

Please notice that such semantic information extracted from these examples may be used to identify other anomalous time series as well.

5) *Hyperparameter Sensitivity Analysis:* In Table IV, we list four hyperparameters of ET-Net. N_E determines the number of tasks in the W compression network, while N_L specifies the number of resolution levels used to parse the time series in the D compression network. To evaluate their impact, we vary N_E and N_L from 1 to 4 and report the average AUC of ET-Net-W and ET-Net-D after five runs on the UNSW-IoT dataset, as depicted in Fig. 13(a). Our findings suggest that increasing both N_E and N_L enhances performance. Datasets with heterogeneous temporal dynamics, such as UNSW-IoT, can benefit from an increased number of Gaussian components K . This adjustment better estimates the complex distribution of the latent space, as indicated by the green line in Fig. 13(a). Moreover, we investigated the effect of the number of neurons N_N on ET-Net's performance. Fig. 13(b) highlights that increasing the number of neurons leads to improved performance.

TABLE VIII
CLUSTERING PERFORMANCE MEASURED BY NMI

Method	UNSW-IoT	IoT23	Cell traffic
K-means	0.0202	0.0399	0.0312
GMM	0.0000	0.0000	0.0107
K-means + DTW	0.5882	0.4860	0.0189
K-means + EDR	0.5046	0.3260	0.0318
K-shape	0.6071	0.4558	0.0258
SPIRAL	0.9138	0.4390	0.0336
Autowarp	0.1002	0.1987	0.0281
DEC	0.0202	0.3091	0.0102
IDEK	0.0201	0.2569	0.0195
DTC	0.6117	0.1862	0.0000
AE + K-means	0.7033	0.6268	0.0269
ET-Net + K-means	0.6701	0.5659	0.0366
ET-Net-W	0.4023	0.6327	0.0542
ET-Net-D	0.3734	0.2694	0.0193
ET-Net	<u>0.8304</u>	0.6753	0.0582

Optimal and suboptimal results are bolded and underlined, respectively.

It is worth noting that the computational complexity of the model depends on the hyperparameters N_L , N_E , and N_N , which have complexities of $O(N_L)$, $O(N_E)$, and $O(N_N^2)$, respectively. In anomaly detection tasks, selecting the appropriate number of Gaussian components K is crucial, and should be determined based on the complexity of the distribution of the normal samples. For small datasets, it is recommended to appropriately reduce these parameters to avoid overfitting. As demonstrated in Fig. 13(b), increasing the number of neurons to 24 results in a decrease in performance. On the other hand, in clustering tasks, the number of GMM components is set equal to the number of clusters.

D. Time Series Clustering Results on Real-World Datasets

Implementation Details: We conduct clustering experiments on three real datasets, including UNSW-IoT, IoT23 and cell traffic. Hyperparameters used in the experiment are listed in Table IV.

Effectiveness: Table VIII lists the clustering performance of ET-Net and other state-of-the-art methods on three real-world datasets. Two other methods have also been considered for comparison, including 1) AE+K-means in which the clustering is carried out over the latent space representations, which is obtained through the sequence autoencoder. 2) ET-Net+K-means in which the clustering is carried out over the vector embeddings obtained by the W and D compression network. For both approaches, the same network hyperparameters as ET-Net are adopted.

The results show that ET-Net outperforms all other state-of-the-art methods in two out of three datasets. This substantiates the effectiveness of the ET-Net for clustering. However, as noted in [70], there is no universally suitable similarity metric for all tasks. SPIRAL, which utilizes the inner product of representations as a similarity metric, outperforms other distance-based methods in the UNSW-IoT dataset. Moreover, the proposed method performs best among all distance-based methods.

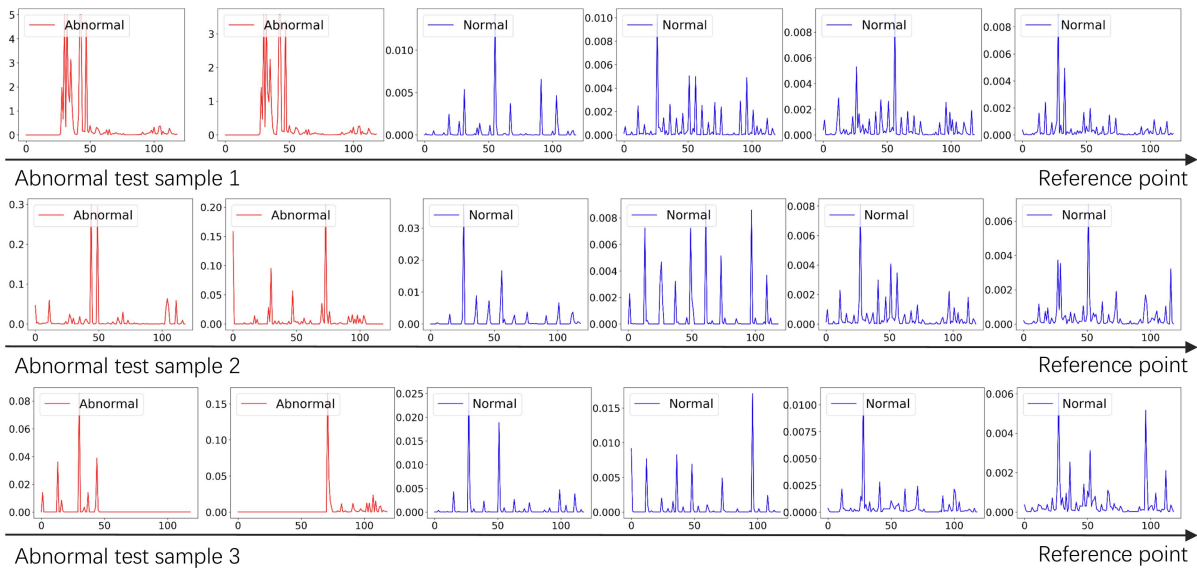


Fig. 14. Exemplary test samples and corresponding reference samples from the UNSW-IoT dataset. The first column is the test samples, and each subsequent column is closer to the center of the normal cluster. The x -axis represents time in minutes and the y -axis represents traffic in kilobytes.

V. CONCLUSION

In this paper, we present ET-Net, an unsupervised deep learning approach that learns similarity metrics on event-triggered time series. Through extensive qualitative and quantitative studies, it is revealed that the proposed model can effectively capture the temporal dynamics of event-triggered time series. In addition, a single ET-Net model can be applied to time series with different time granularity with little performance degradation, which shows its robustness.

REFERENCES

- [1] J. Ren, D. J. Dubois, D. Choffnes, A. M. Mandalari, R. Kolcun, and H. Haddadi, "Information exposure from consumer IoT devices: A multidimensional, network-informed measurement approach," in *Proc. Internet Meas. Conf.*, 2019, pp. 267–279.
- [2] K. Yang, R. Liu, Y. Sun, J. Yang, and X. Chen, "Deep network analyzer (DNA): A Big data analytics platform for cellular networks," *IEEE Internet Things J.*, vol. 4, no. 6, pp. 2019–2027, Dec. 2017.
- [3] M. Villarreal-Vasquez, G. Modelo-Howard, S. Dube, and B. Bhargava, "Hunting for insider threats using LSTM-based anomaly detection," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 451–462, Jan./Feb. 2023.
- [4] F. Simmross-Wattenberg, J. I. Asensio-Perez, P. Casaseca-de-la Higuera, M. Martin-Fernandez, I. A. Dimitriadis, and C. Alberola-Lopez, "Anomaly detection in network traffic based on statistical inference and stable modeling," *IEEE Trans. Dependable Secure Comput.*, vol. 8, no. 4, pp. 494–509, Jul./Aug. 2011.
- [5] I. Nevat et al., "Anomaly detection and attribution in networks with temporally correlated traffic," *IEEE/ACM Trans. Netw.*, vol. 26, no. 1, pp. 131–144, Feb. 2018.
- [6] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1–9.
- [7] A. M. Lamb, A. Goyal, Y. Zhang, S. Zhang, A. C. Courville, and Y. Bengio, "Professor forcing: A new algorithm for training recurrent networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [8] H. Wang, H. Su, K. Zheng, S. Sadiq, and X. Zhou, "An effectiveness study on trajectory similarity measures," in *Proc. 24th Australas. Database Conf.*, 2013, pp. 13–22.
- [9] G. Eibl and D. Engel, "Influence of data granularity on smart meter privacy," *IEEE Trans. Smart Grid*, vol. 6, no. 2, pp. 930–939, Mar. 2015.
- [10] E. Keogh and S. Kasetty, "On the need for time series data mining benchmarks: A survey and empirical demonstration," in *Proc. 8th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2002, pp. 102–111.
- [11] L. Chen, M. T. Özsu, and V. Oria, "Robust and fast similarity search for moving object trajectories," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2005, pp. 491–502.
- [12] H. Sakoe and S. Chiba, "Dynamic programming algorithm optimization for spoken word recognition," *IEEE Trans. Acoust., Speech, Signal Process.*, vol. 26, no. 1, pp. 43–49, Feb. 1978.
- [13] M. Toller, B. C. Geiger, and R. Kern, "A formally robust time series distance metric," in *Proc. 5th Workshop Mining Learn. From Time Ser. (Held Conjunction)*, 2019, pp. 1–10.
- [14] D. F. Silva et al., "On the effect of endpoints on dynamic time warping," in *Proc. SIGKDD Workshop Mining Learn. From Time Ser. II*, 2016, pp. 1–10.
- [15] M. Cuturi and M. Blondel, "Soft-DTW: A differentiable loss function for time series," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 894–903.
- [16] X. Cai, T. Xu, J. Yi, J. Huang, and S. Rajasekaran, "DTWNet: A dynamic time warping network," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 11636–11646.
- [17] A. Mathew et al., "Warping resilient time series embeddings," in *Proc. Time Ser. Workshop 36th Int. Conf. Mach. Learn.*, 2019, pp. 1–5.
- [18] Q. Ma, J. Zheng, S. Li, and G. W. Cottrell, "Learning representations for time series clustering," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 3776–3786.
- [19] A. Abid and J. Y. Zou, "Learning a warping distance from unlabeled time series using sequence autoencoders," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 10547–10555.
- [20] C. Cortes and V. Vapnik, "Support-vector networks," *Mach. Learn.*, vol. 20, no. 3, pp. 273–297, 1995.
- [21] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.
- [22] B. Schölkopf, R. C. Williamson, A. J. Smola, J. Shawe-Taylor, and J. C. Platt, "Support vector method for novelty detection," in *Proc. Adv. Neural Inf. Process. Syst.*, 2000, pp. 582–588.
- [23] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.
- [24] K. Leung and C. Leckie, "Unsupervised anomaly detection in network intrusion detection using clusters," in *Proc. 28th Australas. Conf. Comput. Sci.*, 2005, pp. 333–342.
- [25] P. Baldi, "Autoencoders, unsupervised learning, and deep architectures," in *Proc. ICML Workshop Unsupervised Transfer Learn.*, 2012, pp. 37–49.

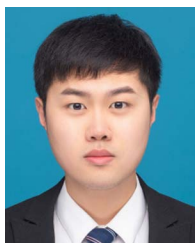
- [26] B. Zhou, S. Liu, B. Hooi, X. Cheng, and J. Ye, "BeatGAN: Anomalous rhythm detection using adversarially generated time series," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 4433–4439.
- [27] L. Shen, Z. Yu, Q. Ma, and J. T. Kwok, "Time series anomaly detection with multiresolution ensemble decoding," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 9567–9575.
- [28] B. Zong et al., "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.
- [29] Y. Zhao, Q. Ding, and X. Zhang, "AE-FLOW: Autoencoders with normalizing flows for medical images anomaly detection," in *Proc. 11st Int. Conf. Learn. Representations*, 2023, pp. 1–5.
- [30] D. Luo, R. Yang, B. Li, and J. Huang, "Detection of double compressed AMR audio using stacked autoencoder," *IEEE Trans. Inf. Forensics Security*, vol. 12, no. 2, pp. 432–444, Feb. 2017.
- [31] L. Yang, N.-M. Cheung, J. Li, and J. Fang, "Deep clustering by gaussian mixture variational autoencoders with graph embedding," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, 2019, pp. 6440–6449.
- [32] P. An, Z. Wang, and C. Zhang, "Ensemble unsupervised autoencoders and gaussian mixture model for cyberattack detection," *Inf. Process. Manage.*, vol. 59, no. 2, pp. 1–13, 2022.
- [33] M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," in *Proc. Eur. Conf. Comput. Vis.*, 2014, pp. 818–833.
- [34] M. T. Ribeiro, S. Singh, and C. Guestrin, "“why should I trust you?” explaining the predictions of any classifier," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 1135–1144.
- [35] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.
- [36] P. Dabkowski and Y. Gal, "Real time image saliency for black box classifiers," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6967–6976.
- [37] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 3319–3328.
- [38] J. V. Jeyakumar, J. Noor, Y.-H. Cheng, L. Garcia, and M. Srivastava, "How can I explain this to you? an empirical study of deep neural network explanation methods," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 1–12.
- [39] P. W. Koh and P. Liang, "Understanding black-box predictions via influence functions," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1885–1894.
- [40] N. Papernot and P. McDaniel, "Deep k-nearest neighbors: Towards confident, interpretable and robust deep learning," 2018, *arXiv: 1803.04765*.
- [41] C. Chen, O. Li, D. Tao, A. Barnett, C. Rudin, and J. K. Su, "This looks like that: Deep learning for interpretable image recognition," in *Proc. Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–12.
- [42] R. Guidotti, A. Monreale, S. Matwin, and D. Pedreschi, "Black box explanation by learning image exemplars in the latent feature space," in *Proc. Mach. Learn. Knowl. Discov. Databases: Eur. Conf.*, 2020, pp. 189–205.
- [43] H. Tahaei, F. Afifi, A. Asemi, F. Zaki, and N. B. Anuar, "The rise of traffic classification in IoT networks: A survey," *J. Netw. Comput. Appl.*, vol. 154, pp. 1–20, 2020.
- [44] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *Proc. Int. Conf. Learn. Representations*, 2013, pp. 1–14.
- [45] V. Fortuin, M. Hüser, F. Locatello, H. Strathmann, and G. Rätsch, "SOM-VAE: Interpretable discrete representation learning on time series," in *Proc. Int. Conf. Learn. Representations*, 2019, pp. 1–18.
- [46] I. Sutskever, O. Vinyals, and Q. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, pp. 1–9, 2014.
- [47] M. Long, Z. Cao, J. Wang, and S. Y. Philip, "Learning multiple tasks with multilinear relationship networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 1594–1603.
- [48] C. Mao et al., "Multitask learning strengthens adversarial robustness," in *Proc. 16th Eur. Conf. Comput. Vis.*, 2020, pp. 158–174.
- [49] T. Kieu, B. Yang, C. Guo, and C. S. Jensen, "Outlier detection for time series with recurrent autoencoder ensembles," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2725–2732.
- [50] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [51] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. Workshop Deep Learn.*, 2014, pp. 1–9.
- [52] S. Chang et al., "Dilated recurrent neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 77–87.
- [53] D. Hendrycks and K. Gimpel, "A Baseline for detecting misclassified and out-of-distribution examples in neural networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–12.
- [54] A. Sivanathan et al., "Classifying IoT devices in smart environments using network traffic characteristics," *IEEE Trans. Mobile Comput.*, vol. 18, no. 8, pp. 1745–1759, Aug. 2019.
- [55] J. Ortiz, C. Crawford, and F. Le, "DeviceMien: Network device behavior modeling for identifying unknown IoT devices," in *Proc. Int. Conf. Internet Things Des. Implementation*, 2019, pp. 106–117.
- [56] A. Sivanathan et al., "Characterizing and classifying IoT traffic in smart cities and campuses," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2017, pp. 559–564.
- [57] S.-E. Benkabou, K. Benabdeslem, and B. Canitia, "Unsupervised outlier detection for time series by entropy and dynamic time warping," *Knowl. Inf. Syst.*, vol. 54, no. 2, pp. 463–486, 2018.
- [58] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," 2018, *arXiv: 1802.09089*.
- [59] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," in *Proc. ICML Anomaly Detection Workshop*, 2016, pp. 1–5.
- [60] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
- [61] J. Paparrizos and L. Gravano, "K-shape: Efficient and accurate clustering of time series," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 1855–1870.
- [62] J. Xie, R. Girshick, and A. Farhadi, "Unsupervised deep embedding for clustering analysis," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 478–487.
- [63] X. Guo, L. Gao, X. Liu, and J. Yin, "Improved deep embedded clustering with local structure preservation," in *Proc. 26th Int. Joint Conf. Artif. Intell.*, 2017, pp. 1753–1759.
- [64] Q. Lei, J. Yi, R. Vaculin, L. Wu, and I. S. Dhillon, "Similarity preserving representation learning for time series clustering," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 2845–2851.
- [65] N. S. Madiraju, S. M. Sadat, D. Fisher, and H. Karimabadi, "Deep temporal clustering: Fully unsupervised learning of time-domain features," 2018, *arXiv: 1802.01059*.
- [66] N. Magdy, M. A. Sakr, T. Mostafa, and K. El-Bahnasy, "Review on trajectory similarity measures," in *Proc. 7th IEEE Int. Conf. Intell. Comput. Inf. Syst.*, 2015, pp. 613–619.
- [67] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv: 1412.6980*.
- [68] X. Cai, R. Nithyanand, T. Wang, R. Johnson, and I. Goldberg, "A systematic approach to developing and evaluating website fingerprinting defenses," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2014, pp. 227–238.
- [69] M. Nasr, A. Bahramali, and A. Houmansadr, "Defeating DNN-based traffic analysis systems in real-time with blind adversarial perturbations," in *Proc. USENIX Secur. Symp.*, 2021, pp. 2705–2722.
- [70] A. S. Shirkhorshidi, S. Aghabozorgi, and T. Y. Wah, "A comparison study on similarity and dissimilarity measures in clustering continuous data," *PLoS One*, vol. 10, no. 12, 2015, Art. no. e0144059.



Shaoyu Dou received the BEng degree from Hohai University, Nanjing, China, in 2018, and the PhD degree from Tongji University, Shanghai, China. She currently holds the position of senior research & development engineer at Ant Group. Her primary research interests include large language models, AI for IT Operations, Big Data analytics, and machine learning.



Kai Yang (Senior Member, IEEE) received the BEng degree from Southeast University, Nanjing, China, the MS degree from the National University of Singapore, Singapore, and the PhD degree from Columbia University, New York, NY, USA. He is a distinguished professor with Tongji University, Shanghai, China. He was a technical staff member with Bell Laboratories, Murray Hill, NJ, USA. He has also been an adjunct faculty member with Columbia University since 2011. He holds more than 20 patents and has been published extensively in leading IEEE journals and conferences. His current research interests include Big Data analytics, machine learning, wireless communications, and signal processing.



Yang Jiao received the BS degree from Central South University, Changsha, China, in 2020. He is currently working toward the PhD degree in computer science with the Department of Computer Science and Technology, Tongji University, Shanghai, China. He has authored at top-tier artificial intelligence conferences, such as NeurIPS, ICLR in his research areas, which include machine learning, robust optimization, and distributed optimization.



Chengbo Qiu received the MS degree from the Huazhong University of Science and Technology, Wuhan, China, in 2018. He is currently working toward the PhD degree in computer science from the Department of Computer Science, Tongji University, Shanghai, China. His major research interests include Big Data analytics and machine learning.



Kui Ren (Fellow, IEEE) received the BEng degree in chemical engineering, in 1998, the MEng degree in materials engineering, in 2001, both from Zhejiang University, China, and the PhD degree in electrical and computer engineering from Worcester Polytechnic Institute, USA, in 2007. Professor Kui Ren, AAAS, ACM, CCF, is currently the dean of the College of Computer Science and Technology, Zhejiang University. He is mainly engaged in research in data security and privacy protection, AI security, and security in intelligent devices and vehicular networks.